Crypto Cheatsheet

Zwaktes, aanvallen en codefragmenten

Opmerking

Gebaseerd op: samenvatting hashing/encryptie + jouw oefeningen (Les 1 t/m 4: stream ciphers, AES/DES, hashing, signing).
Alle fragmenten zijn pure Python (stdlib + `cryptography`-package, geen `pycryptodome` nodig tenzij anders vermeld). Pas sleutels/bytes aan naar jouw eigen opgave.

Contents

1 Basis bouwstenen die overal terugkomen **2**
- 1.1 Bit flippen (XOR met 1 op een positie) . . . 2
- 1.2 XOR van twee gelijke-lengte bytes (en zijn eigen inverse) . . . 2
- 1.3 Hex/bytes/tekst omzettingen (kom je overal tegen) . . . 2
- 1.4 Rotaties (rol/ror) . . . 2

2 Stream ciphers – zwaktes en aanvallen **2**
- 2.1 Caesar / monoalfabetisch . . . 2
- 2.2 Vigenère . . . 3
- 2.3 Keystream/nonce hergebruik (stream cipher of CTR-mode) . . . 3
- 2.4 Custom PRNG/LCG-gebaseerde stream cipher . . . 3
- 2.5 Bit-flipping attack op stream cipher / CTR (vormbaarheid) . . . 4
- 2.6 RC4 . . . 4
- 2.7 Salsa20 . . . 4

3 Block ciphers (DES/AES) en modes – zwaktes **5**
- 3.1 DES: kleine sleutelruimte . . . 5
- 3.2 ECB-mode: patronen blijven zichtbaar . . . 5
- 3.3 Meet-in-the-middle op dubbele encryptie . . . 6
- 3.4 CBC-mode: bitflip/padding attacks . . . 6
- 3.5 CTR-mode: teller/nonce hergebruikt . . . 6

4 Hashfuncties – zwaktes en aanvallen **6**
- 4.1 Custom/zwakke hash reverse-engineeren . . . 7
- 4.2 Brute-force preimage attack . . . 7
- 4.3 Birthday attack / collision zoeken . . . 8
- 4.4 Length extension attack . . . 8
- 4.5 Onsalted password hashes . . . 8
- 4.6 SHA256-ketting voorspellen . . . 9

5 Digest auth / signing / “Signed and Sealed” – zwaktes **9**
- 5.1 HTTP/RTSP Digest Authentication . . . 9
- 5.2 “Signed” JSON-berichten die je ongewijzigd doorstuurt . . . 9

6 Algemene checklist bij een nieuwe opgave **10**

1 Basis bouwstenen die overal terugkomen

1.1 Bit flippen (XOR met 1 op een positie)

```python
def flip_bit(data: bytes, bit_index: int) -> bytes:
    """Flip 1 bit in een bytes-object. bit_index telt vanaf MSB van byte 0."""
    byte_index = bit_index // 8
    bit_in_byte = 7 - (bit_index % 8)
    b = bytearray(data)
    b[byte_index] ^= (1 << bit_in_byte)
    return bytes(b)
```

1.2 XOR van twee gelijke-lengte bytes (en zijn eigen inverse)

```python
def xor_bytes(a: bytes, b: bytes) -> bytes:
    return bytes(x ^ y for x, y in zip(a, b))

# XOR is zijn eigen inverse: a ^ b ^ b == a
# Daarom: als je 2 van de 3 waarden hebt (pt, ct, keystream) ken je de derde.
#   ct = pt ^ keystream   ->   keystream = pt ^ ct   ->   pt = ct ^ keystream
```

1.3 Hex/bytes/tekst omzettingen (kom je overal tegen)

```python
h = "48656c6c6f"
b = bytes.fromhex(h)           # hex -> bytes
back = b.hex()                  # bytes -> hex
s = b.decode("utf-8", errors="ignore")   # bytes -> tekst (fouttolerant)
b2 = "Hello".encode("utf-8")   # tekst -> bytes  (zelfde als bytes(s,'utf-8'))
```

1.4 Rotaties (rol/ror)

Komen voor bij custom hash/cipher-ontwerp.

```python
def rol8(x, n):   # rotate-left binnen 1 byte (8 bits)
    n %= 8
    return ((x << n) | (x >> (8 - n))) & 0xff

def ror8(x, n):   # rotate-right binnen 1 byte
    n %= 8
    return ((x >> n) | (x << (8 - n))) & 0xff
```

2 Stream ciphers – zwaktes en aanvallen

2.1 Caesar / monoalfabetisch

Probleem

Kleine sleutelruimte.

Oplossing / aanval

Brute-force alle sleutels, check op leesbare ASCII.

```python
def caesar_brute_force(ciphertext: str):
    for shift in range(26):
        pt = "".join(
            chr((ord(c) - 65 - shift) % 26 + 65) if c.isupper()
            else chr((ord(c) - 97 - shift) % 26 + 97) if c.islower()
            else c
```

```
            for c in ciphertext
        )
        print(shift, pt)
```

2.2 Vigenère

Probleem

Herhaalde sleutel → patronen in de ciphertext.

Oplossing / aanval

Sleutellengte zoeken (Kasiski / index of coincidence), dan per kolom frequentie-analyse (elke kolom = Caesar). Voor een les-opgave is brute-force van korte sleutels vaak sneller.

```
def vigenere_decrypt(ciphertext: str, key: str) -> str:
    out = []
    ki = 0
    for c in ciphertext:
        if c.isalpha():
            base = 65 if c.isupper() else 97
            k = ord(key[ki % len(key)].lower()) - 97
            out.append(chr((ord(c) - base - k) % 26 + base))
            ki += 1
        else:
            out.append(c)
    return "".join(out)
```

2.3 Keystream/nonce hergebruik (stream cipher of CTR-mode)

Probleem

Keystream/nonce hergebruik bij stream cipher of CTR-mode (zie Les 2/Noncense – exact dit scenario!).

Oplossing / aanval

Als 2 berichten met dezelfde keystream versleuteld zijn: $c_1 \oplus c_2 = p_1 \oplus p_2$. Als je 1 plaintext kent, ken je de keystream voor beide, dus ook het 2e bericht.

```
def recover_via_nonce_reuse(known_ct: bytes, known_pt: bytes, target_ct: bytes) -> bytes:
    """known_ct/known_pt = bericht waarvan je BEIDE kent.
       target_ct = het bericht dat je wil kraken (zelfde key+nonce)."""
    keystream = xor_bytes(known_ct, known_pt)
    # alleen zoveel bytes als beschikbaar/nodig gebruiken:
    n = min(len(target_ct), len(keystream))
    return xor_bytes(target_ct[:n], keystream[:n])
```

Opmerking

Let op (fout die in de oefening stond):

```
#   keystream = xor(known_ct, flag_ct)   <-- FOUT, dit is niet de keystream!
#   Het moet: keystream = xor(known_ct, known_pt)   (ciphertext XOR de ECHTE plaintext)
```

2.4 Custom PRNG/LCG-gebaseerde stream cipher

Probleem

Les 1/Replanting. State-update: `state = (A*state + C) mod M`; keystream-byte = `(state » 16) & 0xff`.

Oplossing / aanval

Zwakte: LCG is voorspelbaar/omkeerbaar zodra je enkele output-bytes kent (en A, C bekend zijn). Seed/state kan teruggerekend of gebruteforced worden (zeker als alleen de lage bits onbekend zijn).

```
A, C, M = 1103515245, 12345, 2**31

def lcg_next(state):
    return (A * state + C) % M

def lcg_keystream(seed, length):
    state = seed
    ks = bytearray()
    for _ in range(length):
        state = lcg_next(state)
        ks.append((state >> 16) & 0xff)
    return bytes(ks)

def brute_force_seed(known_pt: bytes, known_ct: bytes, seed_bits=24):
    """Bruteforce een kleine seed-ruimte (bv. 2^24) met een snelle
    voorfilter op de eerste paar bytes."""
    target_ks = xor_bytes(known_pt, known_ct)
    for seed in range(2 ** seed_bits):
        if lcg_keystream(seed, 4) == target_ks[:4]:          # snelle filter
            if lcg_keystream(seed, len(target_ks)) == target_ks:  # volledige check
                return seed
    return None
```

2.5 Bit-flipping attack op stream cipher / CTR (vormbaarheid)

Probleem

Stream cipher/CTR is vormbaar (malleable).

Oplossing / aanval

Je kan een exact gekozen plaintext-wijziging doorvoeren zonder de sleutel te kennen, door op de juiste positie in de ciphertext te XOR'en.

```
def bitflip_change_plaintext(ciphertext: bytes, old_plain: bytes, new_plain: bytes, offset: int = 0) -> bytes:
    """Verandert old_plain naar new_plain op 'offset' in de ciphertext,
       old_plain en new_plain moeten gelijke lengte hebben."""
    assert len(old_plain) == len(new_plain)
    patch = xor_bytes(old_plain, new_plain)
    ct = bytearray(ciphertext)
    for i, p in enumerate(patch):
        ct[offset + i] ^= p
    return bytes(ct)
```

2.6 RC4

Probleem

RC4 keystream-bias / RC4 gebruiken.

Oplossing / aanval

Zwakte is structureel (statistische bias in eerste bytes), dus: niet zelf oplossen met code, gewoon RC4 vermijden / melden als kwetsbaarheid.

2.7 Salsa20

Probleem

Zie Les 1/Salsa20-map.

Oplossing / aanval

Zelfde principe als elke stream cipher: hergebruik van (key, nonce) is fataal. Met pycryptodome:

```
#   from Crypto.Cipher import Salsa20
#   cipher = Salsa20.new(key=key, nonce=nonce)
```

```
#   ct = cipher.encrypt(pt)
```

Met de `cryptography`-package (indien pycryptodome niet beschikbaar is):

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms

def salsa20_encrypt(key: bytes, nonce: bytes, data: bytes) -> bytes:
    algo = algorithms.ChaCha20(key, nonce)  # ChaCha20 ipv Salsa20 (zelfde familie, cryptography-lib heeft geen
    ↪ Salsa20)
    c = Cipher(algo, mode=None)
    return c.encryptor().update(data)
```

3 Block ciphers (DES/AES) en modes – zwaktes

3.1 DES: kleine sleutelruimte

Probleem

DES-sleutel van maar 56 bits (of klein keyspace door slecht gekozen sleutel, zie Les 2/DES.py: "IOTDEV" + 2 cijfers).

Oplossing / aanval

Brute-force alle kandidaat-sleutels, check op leesbare output.

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

def des_ecb_bruteforce(ciphertext: bytes, namespace: str, max_suffix=100):
    for i in range(max_suffix):
        candidate_key = (namespace + f"{i:02d}").encode("ascii")
        # DES key moet 8 bytes zijn
        if len(candidate_key) != 8:
            continue
        c = Cipher(algorithms.TripleDES(candidate_key * 1), modes.ECB())  # of algorithms.DES als lib dat biedt
        try:
            dec = c.decryptor()
            pt = dec.update(ciphertext) + dec.finalize()
            if all(32 <= b < 127 for b in pt[:16]):
                print("key:", candidate_key, "->", pt)
                return candidate_key, pt
        except Exception:
            continue
    return None, None
```

3.2 ECB-mode: patronen blijven zichtbaar

Probleem

ECB-mode: identieke plaintext-blokken → identieke ciphertext-blokken (patronen blijven zichtbaar, "ECB-penguin").

Oplossing / aanval

Herken het door te kijken of blokken (bv. 16 bytes bij AES) zich herhalen in de ciphertext:

```
def find_repeated_blocks(data: bytes, block_size=16):
    blocks = [data[i:i+block_size] for i in range(0, len(data), block_size)]
    seen = {}
    repeats = []
    for idx, blk in enumerate(blocks):
        if blk in seen:
            repeats.append((seen[blk], idx, blk.hex()))
        else:
            seen[blk] = idx
    return repeats  # niet-leeg = sterk signaal voor ECB + patroonlek
```

3.3 Meet-in-the-middle op dubbele encryptie

Probleem

Dubbele encryptie (2x AES met korte deelsleutels, zie Les 2/AES@): 2x 20-bit sleutel breek je **niet** door 2^{40} te doorzoeken.

Oplossing / aanval

Wel door 2×2^{20} (meet-in-the-middle):

```python
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

def make_key(k: int) -> bytes:
    return k.to_bytes(3, "big") + b"\x00" * 13   # pad naar 16 bytes AES-128

def aes_ecb_encrypt_block(key: bytes, block: bytes) -> bytes:
    c = Cipher(algorithms.AES(key), modes.ECB()).encryptor()
    return c.update(block) + c.finalize()

def aes_ecb_decrypt_block(key: bytes, block: bytes) -> bytes:
    c = Cipher(algorithms.AES(key), modes.ECB()).decryptor()
    return c.update(block) + c.finalize()

def meet_in_the_middle(known_pt: bytes, known_ct: bytes, key_bits=20):
    table = {}
    for k1 in range(2 ** key_bits):
        mid = aes_ecb_encrypt_block(make_key(k1), known_pt)
        table[mid] = k1
    for k2 in range(2 ** key_bits):
        mid = aes_ecb_decrypt_block(make_key(k2), known_ct)
        if mid in table:
            return table[mid], k2
    return None
```

```python
# Gebruik: zoek een bekend plaintext-blok (bv. vaste header van een
# bestandsformaat zoals BMP-pixel masks) en het bijhorende ciphertext-blok,
# draai dan meet_in_the_middle(). Daarna: plain = D_k1(D_k2(ciphertext)).
```

3.4 CBC-mode: bitflip/padding attacks

Probleem

CBC-mode: bits in het vorige blok aanpassen → voorspelbare wijziging in het volgende blok (padding/bit-flip attacks).

Oplossing / aanval

Zelfde principe als `bitflip_change_plaintext()` hierboven, maar dan toegepast op het **vorige** ciphertext-blok (niet het blok zelf), omdat CBC het vorige blok XOR't met de huidige plaintext vóór encryptie.

3.5 CTR-mode: teller/nonce hergebruikt

Probleem

CTR-mode: teller/nonce hergebruikt.

Oplossing / aanval

Zelfde als "keystream reuse" hierboven (CTR = stream cipher!). Gebruik `recover_via_nonce_reuse()`.

4 Hashfuncties – zwaktes en aanvallen

4.1 Custom/zwakke hash reverse-engineeren

Probleem

Custom/zwakke hash (Les 3/better hash).

Oplossing / aanval

Als elke ronde-operatie omkeerbaar is (XOR, rotate, shift), kan je de hash gewoon in omgekeerde volgorde en met inverse operaties terugrekenen. Patroon: bouw de encrypt-functie na, keer dan elke stap om (laatste stap eerst, elke operatie met zijn inverse):

- XOR → inverse is dezelfde XOR
- shift/rotate links → inverse is rotate rechts (en vice versa)
- volgorde van rondes → ook omkeren (laatste ronde eerst)

```
def rol(x, n):
    return ((x << n) | (x >> (8 - n))) & 0xff

def ror(x, n):
    return ((x >> n) | (x << (8 - n))) & 0xff

def forward_hash(msg: str, rounds=8) -> str:
    state = bytearray(msg.encode())
    for rnd in range(rounds):
        for i in range(len(state)):
            state[i] ^= (rnd * 41 + i * 17) & 0xff
            state[i] = rol(state[i], (rnd + i) & 7)
        for i in range(1, len(state)):
            state[i] ^= state[i - 1]
        state = state[1:] + state[:1]       # rotate de hele array 1 plek naar links
    return state.hex()

def reverse_hash(hex_digest: str, rounds=8) -> bytes:
    state = bytearray.fromhex(hex_digest)
    for rnd in range(rounds - 1, -1, -1):
        state = state[-1:] + state[:-1]     # inverse van "rotate array links" = rotate rechts
        for i in range(len(state) - 1, 0, -1):
            state[i] ^= state[i - 1]        # XOR-chain terugdraaien (van achter naar voor!)
        for i in range(len(state) - 1, -1, -1):
            state[i] = ror(state[i], (rnd + i) & 7)   # inverse van rol = ror
            state[i] ^= (rnd * 41 + i * 17) & 0xff
    return bytes(state)

# Tip: check altijd of forward_hash(reverse_hash(x)) == x om je inverse-logica
# te verifieren voor je op een echte flag loslaat.
```

4.2 Brute-force preimage attack

Probleem

Preimage: 2^{m-1} pogingen voor een m-bit hash.

Oplossing / aanval

Praktisch alleen haalbaar bij kleine hash (bv. 32–64 bit) of korte bekende invoerruimte (bv. 6-cijferige pincode):

```
import hashlib

def bruteforce_preimage(target_hex: str, charset="0123456789", length=6, algo="sha256"):
    import itertools
    h = getattr(hashlib, algo)
    for combo in itertools.product(charset, repeat=length):
        candidate = "".join(combo).encode()
        if h(candidate).hexdigest() == target_hex:
            return candidate
    return None
```

4.3 Birthday attack / collision zoeken

Probleem

Collision zoeken: $2^{m/2}$ i.p.v. 2^m pogingen – zie Les 3/Banking2/collision.json: 2 verschillende "reference"-strings, zelfde fingerprint/hash.

Oplossing / aanval

Genereer veel varianten en zoek op in een hashtabel (dict) tot een botsing optreedt:

```
import hashlib, os

def find_collision(prefix: str, suffix_len=8, algo="md5"):
    h = getattr(hashlib, algo)
    seen = {}
    while True:
        candidate = prefix + os.urandom(suffix_len).hex()
        digest = h(candidate.encode()).hexdigest()
        if digest in seen and seen[digest] != candidate:
            return seen[digest], candidate, digest
        seen[digest] = candidate
```

4.4 Length extension attack

Probleem

$H(\text{key} \,\|\, \text{message})$ als MAC gebruikt; MD5/SHA-1/SHA-2 zijn kwetsbaar.

Oplossing / aanval

Met de bekende digest + lengte van de key kan je data achteraan plakken en een geldige nieuwe hash berekenen zonder de key te kennen. Gebruik een bestaande library hiervoor (zelf Merkle-Damgard padding nabouwen is foutgevoelig); bv. `hashpumpy` (pip) of `hlextend` (pip). Mini-voorbeeld met `hlextend` (indien geïnstalleerd):

```
# import hlextend
# sha = hlextend.new('sha256')
# new_sig = sha.extend(b";admin=true", known_message, key_length, known_digest)
# forged_message = sha.generate()
#
# Oplossing voor de ontwikkelaar: gebruik HMAC (hashlib.new werkt niet als
# MAC-vervanger) of SHA-3 (sponsconstructie is niet kwetsbaar).
```

```
import hmac, hashlib

def correct_mac(key: bytes, message: bytes) -> str:
    return hmac.new(key, message, hashlib.sha256).hexdigest()   # VEILIG, geen length-extension
```

4.5 Onsalted password hashes

Probleem

Onsalted password hashes → rainbow tables / dictionary attack. Herkenning: zelfde wachtwoord geeft altijd dezelfde hash zonder salt.

Oplossing / aanval

(Voor ontwikkelaars) salt + trage hash (bcrypt/scrypt/argon2), **nooit** kale `sha256(password)`.

```
import hashlib, os

def hash_password_properly(password: str):
    salt = os.urandom(16)
    # scrypt is beschikbaar in stdlib (hashlib.scrypt), traag by design
    digest = hashlib.scrypt(password.encode(), salt=salt, n=2**14, r=8, p=1, dklen=32)
    return salt, digest
```

```python
def verify_password(password: str, salt: bytes, digest: bytes) -> bool:
    candidate = hashlib.scrypt(password.encode(), salt=salt, n=2**14, r=8, p=1, dklen=32)
    return hmac.compare_digest(candidate, digest)   # timing-safe vergelijken!
```

4.6 SHA256-ketting voorspellen

Probleem

Les 3/oefening 3: `next_code = sha256(current_code)`.

Oplossing / aanval

Als je de "huidige" waarde kent, is de "volgende" gewoon 1 hash-call:

```python
import hashlib

def next_code_from_current(current_hex: str) -> str:
    return hashlib.sha256(bytes.fromhex(current_hex)).hexdigest()

# Zwakte hier: als het protocol een voorspelbare hashketting gebruikt
# voor eenmalige codes (OTP-achtig) zonder extra geheim/salt, kan
# iedereen die de huidige waarde kent de volgende voorspellen.
```

5 Digest auth / signing / "Signed and Sealed" – zwaktes

5.1 HTTP/RTSP Digest Authentication

Probleem

MD5-gebaseerd, Les 3/SecurityCam. Zelf geen directe "break", maar ken het stramien:

- `HA1 = MD5(user:realm:pass)`
- `HA2 = MD5(method:uri)`
- `response = MD5(HA1:nonce:HA2)`

Oplossing / aanval

Zwaktes: MD5 is verouderd (collision-gevoelig), en als `pass` zwak is kan je met bekende realm/nonce/method/uri offline wachtwoorden bruteforcen door de response te vergelijken:

```python
import hashlib

def md5(s: str) -> str:
    return hashlib.md5(s.encode()).hexdigest()

def bruteforce_digest_password(username, realm, method, uri, nonce, target_response, wordlist):
    ha2 = md5(f"{method}:{uri}")
    for pw in wordlist:
        ha1 = md5(f"{username}:{realm}:{pw}")
        resp = md5(f"{ha1}:{nonce}:{ha2}")
        if resp == target_response:
            return pw
    return None
```

5.2 "Signed" JSON-berichten die je ongewijzigd doorstuurt

Probleem

Les 4/SignedSealed: message + signature van de server. Veelvoorkomende valkuil: de server signeert de **exacte bytes** van het bericht (bv. een specifieke JSON-serialisatie met bepaalde spaties/sleutelvolgorde). Als jij het bericht zelf opnieuw serialiseert (`json.dumps` met andere volgorde/spacing), klopt de signature niet meer.

Oplossing / aanval

Stuur de RAW bytes/tekst die je ontving door, verander niets:

```python
import requests

def forward_raw_signed_message(base_url: str):
    r = requests.get(f"{base_url}/challenge")
    raw_body = r.text                       # exacte bytes zoals ontvangen, NIET r.json() + json.dumps()
    challenge = r.json()
    signature = challenge["signature"]
    # stuur het 'message'-deel als RAW substring terug i.p.v. opnieuw dumpen:
    import re
    # pak het stuk "message": {...} letterlijk uit de ruwe tekst als dat nodig is,
    # of stuur gewoon raw_body ongewijzigd door als dat is wat verwacht wordt.
    r2 = requests.post(
        f"{base_url}/vault",
        data=raw_body,  # exacte originele bytes, voorkomt serialisatie-mismatch
        headers={"Content-Type": "application/json", "X-Signature": signature},
    )
    return r2.status_code, r2.text
```

Opmerking

Denkwijze: als een signature/hash niet klopt terwijl de "inhoud" logisch hetzelfde is, verdenk altijd eerst de EXACTE bytes/encoding (spaties, key-volgorde, newline aan einde, utf-8 vs latin-1, trailing `\r\n`), niet de cryptografie zelf.

6 Algemene checklist bij een nieuwe opgave

1. **Wat ken je?** Plaintext, ciphertext, key, nonce, hash-output?
2. **Wat is het cijfer/hash-type?** Stream/block/hash – kijk naar sleutelgrootte, blokgrootte, aantal rondes.
3. **Is er hergebruik?** Zelfde key+nonce op meerdere berichten → XOR-aanval.
4. **Is de sleutelruimte klein?** Vaste prefix/patroon in de sleutel → brute-force i.p.v. volledige keyspace doorzoeken.
5. **Is het dubbele encryptie met korte deelsleutels?** → meet-in-the-middle i.p.v. 2x volledige brute-force.
6. **Is het een custom/eigen hash of cipher?** → elke operatie is meestal omkeerbaar (XOR/rotate/shift), bouw de inverse-functie stap voor stap, in omgekeerde volgorde.
7. **Wordt een hash gebruikt als MAC ($H(\mathbf{key} \,\|\, \mathbf{msg})$)?** → check op length extension attack.
8. **Werkt een signature/hash plots niet terwijl de data "hetzelfde" is?** → verdenk exacte bytes/serialisatie, niet de cryptografie zelf.
9. **Test je code altijd eerst** op het voorbeeld waarvan je het antwoord al kent (round-trip: `encrypt(decrypt(x)) == x`), voor je hem op de echte/onbekende data loslaat.